```
  rspec.after(:example, pry: true) do |ex|
    require 'pry'
    binding.pry
  end
end
```

Each of these aliases will add the pry: true metadata to its respective example group or single example. The after hook then calls binding.pry only after examples that have the :pry metatada defined.

Now, you can quickly toggle the Pry behavior on or off by adding or removing a p at the beginning of describe or it.

Every technique you've seen in this section has been about *clarity*—specifically, about organizing your examples so that they read logically. We're going to continue this theme of clarity in the next section. You'll see how to keep common setup routines from cluttering up the content of your specs.

Sharing Common Logic

Over the past several chapters, you've used several different techniques to share common setup logic.

The main three organization tools are let definitions, hooks, and helper methods. Here's a snippet that contains all three of these items side by side. It's a stripped-down version of the API specs you wrote for *Testing in Isolation: Unit Specs*, including a minor refactoring from the exercises.

07-structuring-code-examples/03/expense_tracker/spec/api_spec.rb

```
RSpec.describe 'POST a successful expense' do
  # let definitions
  let(:ledger)  { instance_double('ExpenseTracker::Ledger') }
  let(:expense) { { 'some' => 'data' } }

  # hook
  before do
    allow(ledger).to receive(:record)
      .with(expense)
      .and_return(RecordResult.new(true, 417, nil))
  end

  # helper method
  def parsed_last_response
    JSON.parse(last_response.body)
  end
end
```

As you've worked through the examples in this book, you've used let definitions several times in your specs. They're great for setting up anything that can be